Employing Metaprogramming: A Macro-Driven Strategy to Overcome Compiler-Imposed Limitations on Specialization

Davide Cerato
Id. Number: 984481

BSc in Computer Science

Advisor: Prof. Walter Cazzola
Co-Advisor: Mr. Federico Cristiano Bruzzone





UNIVERSITÀ DEGLI STUDI DI MILANO
Computer Science Department
ADAPT-Lab

Academic Year 2024–2025

COMPUTER SCIENCE DEPARTMENT • ADAPT-LAB

Contents

1 Introduction **1**

2 Running Example **3**

3 Background **7**
3.1 The Rust Programming Language. 7
3.2 Dispatch Techniques 9
3.3 Metaprogramming 11

4 A deep dive into MyTool **13**
4.1 Introduction 13
4.2 Conditions Definition 14
4.3 Method Wrapping 18
4.4 Build Script 20
4.5 Lifetimes 21
4.6 Limitations 23

5 Experimentation **25**
5.1 Introduction 25
5.2 Results 29

6 Related Work **33**

7 Future Work **35**

8 Conclusions **37**

1 Introduction

Premise A core challenge in high-performance systems programming is achieving zero-cost abstraction through generic programming and polymorphism while allowing type-specific performance optimizations. This concept is known as *specialization* and it's a common aspiration in advanced type systems. This challenge is particularly acute in Rust, a modern systems programming language designed for safety, concurrency and performance. Its trait system enable polymorphism and generic programming without incurring runtime overhead, as the compiler typically performs monomorphization, ensuring high performance across the board. However, for maximum efficiency, there are instances where a generic function could benefit significantly from a custom, highly optimized implementation when dealing with specific types / conditions. The ability to provide such a conditional, type-specific implementation for a trait method is known as Trait Specialization.

State of the Art Many programming languages have explored various mechanisms to achieve specialization, with varying degrees of success and complexity. Without native support for specialization, in many cases the common practice is to rely on various complex workarounds which often result in cumbersome code, poor compiler diagnostics or insufficient optimization compared to true specialization. In 2015, the Rust community introduced an experimental feature called *trait specialization* [37], which soon received widespread interest, but up to now is still confined to the *Nightly* channel, after its stabilization attempts stalled due to various design and implementation challenges.

Limitations A general obstacle for adopting powerful, cutting-edge language features is the tension between innovation and stability, often manifested in experimental versus production-ready compiler versions. This is true for specialization as well, forcing library authors and application developers to make an undesirable trade-off: either forgo the performance and elegance improvements offered by specialization to maintain stability or commit to volatile and constantly changing unstable versions. For example, the primary and most critical limitation facing the Rust ecosystem is indeed the Nightly-Stable divide regarding specialization (the difference between these channels is explained in Section 3.1). While the feature offers significant potential for optimization and cleaner trait designs, its unstable status prevents its adoption in commercial, production-ready or long-term supported codebases.

Problem Statement While the concept of specialization is a crucial tool for optimizing generic code in any performance-oriented language, its practical application is often gated by compiler maturity and stability concerns. The lack or instability of specialization in programming languages creates a gap between what is theoretically possible for optimization and what is practically available to the vast majority of the community, putting a performance ceiling on stable code. Specifically for Rust, given its performance focus and the significant utility of trait specialization, the central problem is the lack of a robust, idiomatic and performance-preserving mechanism for implementing trait specialization in stable code.

Proposal To address the general limitations of specialization features in current stable ecosystems, this thesis proposes an innovative approach to the matter by moving the monomorphization and specialization logics to an earlier stage in the compilation process. To prove the effectiveness of this approach, we introduce a metaprogramming-based technique to simulate the behavior of compiler-implemented specialization. By utilizing the metaprogramming capabilities of the language, we can analyze the input and generate the necessary boilerplate and type-checking logic at compile time, effectively allowing a user to define specialized implementations in stable code, avoiding runtime overhead entirely and circumventing the restrictions imposed by unstable compiler features. This approach can be applied to most languages with metaprogramming support and even directly into the compiler frontend of modern languages, as the compiler can simply replicate our approach internally.

Contributions This work provides the following key contributions. A Novel Stable Specialization Technique: the design and implementation of a new, purely metaprogramming-based approach for achieving specialization. Implementation of Reusable Tools: the creation of a Rust procedural macro library that provides an alternative to nightly specialization and an analysis tool that evaluates real-world public Rust codebases to identify common patterns where this stable specialization mechanism could yield substantial benefits.

Structure The remainder of this thesis is structured as follows. Chapter 2 (Running Example) illustrates the tool in action through a concrete example. Chapter 3 (Background) provides foundational knowledge on Rust, dispatch techniques and metaprogramming. Chapter 4 (A deep dive into MyTool) details the design and architecture of the specialization macro tool. Chapter 5 (Experimentation) presents the experimentation methodology and results, including benchmarks and ecosystem analysis. Chapter 6 (Related Work) reviews existing solutions for specialization in Rust and related languages. Chapter 7 (Future Work) outlines potential extensions and further research avenues. Finally, Chapter 8 (Conclusions) summarizes the findings and concludes the thesis.

2
Running Example

The simplest form of specialization using the tool is the following. We'll show the example in Rust, but the approach is general and can be applied to other languages with similar features.

Let's consider a trait `MyTrait` with a method `my_method`, and we want to have a generic implementation as well as provide a specialized implementation for the type `i32`. In standard Rust this would look something like Listing 1, but unfortunately this code would not compile due to the lack of stable specialization features.

```
1  trait MyTrait {
2      fn my_method(&self);
3  }
4  impl<T> MyTrait for T {
5      fn my_method(&self) {
6          println!("Generic implementation");
7      }
8  }
9  impl MyTrait for i32 {
10     fn my_method(&self) {
11         println!("Specialized for i32");
12     }
13 }
14 fn main() {
15     10.my_method();   // should print "Specialized for i32"
16     10.5.my_method(); // should print "Generic implementation"
17 }
```

Listing 1. *Example of how specialization should work.*

To apply our tool to this problem, the default implementation does not need any change compared to standard code. The specialized implementation instead, is annotated with the attribute macro `#[when(...)]`, that lets us specify the conditions under which that implementation should be used. In this case, we want to specialize the implementation of `MyTrait` for the type `i32`, so we write `#[when(T = i32)]`. This will create a specialized version of the `trait` and of the `impl`, through a process we'll be calling *monomorphic specialization*.

After doing this, we also need to instruct the method calls to use the correct version of the method. This is done by using the `spec!` macro, which takes as first argument the method call and as second argument the type to specialize for. As better explained in Chapter 4, this is required because in the macro expansion phase the compiler doesn't

give us enough information to automatically determine the types of the variables.

The complete code is shown in Listing 2, where you can see the macros in action. As you can probably understand, this approach preserves the soundness of the type system, since after the monomorphization process (see Listing 3) we have two completely separate versions of the trait and its implementation, that the compiler can easily distinguish and type check independently.

```
trait MyTrait {
    fn my_method(&self);
}
impl<T> MyTrait for T {
    fn my_method(&self) {
        println!("Generic implementation");
    }
}
#[when(T = i32)]
impl<T> MyTrait for T {
    fn my_method(&self) {
        println!("Specialized for i32");
    }
}
fn main() {
    spec! { 10.my_method(); i32 };   // prints "Specialized for i32"
    spec! { 10.5.my_method(); f64 }; // prints "Generic implementation"
}
```

Listing 2. *Example of the tool usage.*

```
trait MyTrait {
    fn my_method(&self);
}
impl<T> MyTrait for T {
    fn my_method(&self) {
        println!("Generic implementation");
    }
}
trait MyTrait_i32 {
    fn my_method(&self);
}
impl MyTrait_i32 for i32 {
    fn my_method(&self) {
        println!("Specialized for i32");
    }
}
fn main() {
    <i32 as MyTrait_i32>::my_method(&10); // prints "Specialized for i32"
    <f64 as MyTrait>::my_method(&10.5);   // prints "Generic implementation"
}
```

Listing 3. *Example of the code after our macros' expansion.*

In addition to using the two macros, we need to instruct the compiler to allow us to

define and use the macros correctly independently from the order of their definition. In Rust this is done with *build scripts*, as shown in Listing 4.

```
fn main() {
    spec_trait_order::handle_order();
}
```

Listing 4. *Example of the minimal content of the* `build.rs` *file.*

With this simple example, we have shown how to use the tool to define a specialized implementation of a trait method for a specific type. In the next chapters, we will explore more complex scenarios, including multiple specializations, constraints on generic parameters and how the tool handles these cases internally.

3 Background

3.1 The Rust Programming Language.

Rust is a system programming language that focuses on safety, speed and concurrency. It ensures memory safety without garbage collection, meaning pure Rust programs are free from null pointer dereferences and data races.

Ownership and Borrowing Rust's ownership system—integrated into its type system—is inspired by *linear logic* [19, 20] and *linear types* [47, 35], enforcing that each piece of memory (a value) has a single *owner* (the variable binding) at any time [12, 5]. When the owner goes out of scope, the memory is automatically deallocated, enabling user-defined destructors and supporting the *resource acquisition is initialization* (RAII) pattern [43]. Ownership can be transferred (*moved*) or temporarily shared (*borrowed*) through references. Rust supports two types of borrows: multiple *immutable* borrows or a single *mutable* borrow, but never both at the same time. These constraints, enforced by the compiler through *borrow checking*, guarantee memory safety and prevent dangling pointers by ensuring that reference lifetimes never outlive their owners. To support low-level operations, Rust provides `unsafe` blocks, where the compiler's safety guarantees are suspended and the burden of avoiding undefined behavior falls on the programmer. Outside of these blocks, Rust enforces strict safety, making undefined behavior impossible in safe code.

Structs, Traits and Impls Structs (`struct`) and enums (`enum`) are the primary composite data types in Rust, enabling the creation of complex data structures with named fields and variant types, respectively.

```
struct MyStruct {
    field1: i32,
    field2: String,
}
```

Listing 5. *Example of a Rust struct.*

```
enum MyEnum {
    Variant1,
    Variant2(i32),
}
```

Listing 6. *Example of a Rust enum.*

Rust's trait system is a central mechanism for expressing polymorphism and shared behavior across types [26, 31]. Traits (`trait`) define a set of method signatures that types can implement, enabling abstraction over operations without committing to a concrete type. Implementation blocks (`impl`) associate specific types with traits, providing

concrete method definitions that satisfy the trait's contract. Rust also supports *inherent implementations*, where methods are defined directly on a type without requiring a trait, allowing type-specific functionality and constructor-like associated functions.

```rust
trait MyTrait { fn do_something(&self) }
impl MyTrait for i32 {
    fn do_something(&self) {}
}
```

Listing 7. *Example of a Rust trait impl.*

```rust
struct MyStruct;
impl MyStruct {
    fn do_something(&self) {}
}
```

Listing 8. *Example of a Rust inherent impl.*

By combining *traits*, *trait bounds* and *impl blocks*, Rust achieves ad hoc polymorphism [6, 47] and generic programming [34, 3] while preserving static guarantees and enabling the compiler to resolve method calls efficiently at compile time. Trait-based design is tightly integrated with Rust's ownership and borrowing model, ensuring that polymorphic abstractions do not compromise memory safety.

Software Variability Rust supports software variability through its attribute system, particularly the `cfg` attribute, which allows conditional compilation based on specified configuration predicates A feature in Rust is defined by all *terms* annotated with the same *config name* in a `cfg` *outer attribute*. The inclusion of a feature in a product depends on the evaluation of a *configuration predicate* at compile time, e.g. `#[cfg(any(unix, windows))]`. *Cargo*, Rust's package manager, lets developers declare features using *config names* in the `Cargo.toml` file, combining them with logical operators into *configuration predicates*. Feature dependencies, i.e., *cross-tree constraints* of *feature models* [24, 41, 25], can also be specified in the same file.

Release Channels Rust is distributed through three release channels: *stable*, *beta* and *nightly*. These channels differ in maturity guarantees and access to experimental language features [40]. The *stable* channel prioritizes reliability and backwards compatibility: only fully tested features are enabled, ensuring that libraries and applications compiled on stable continue to work across future releases. The *beta* channel serves as a staging ground for upcoming stable releases, allowing developers to test new features before they reach stable. In contrast, the *nightly* channel is updated daily and exposes experimental compiler internals, unstable language constructs and opt-in features (guarded by `#![feature(...)]` attributes). These features include advanced compiler hooks, low-level intrinsics and incomplete language extensions such as *specialization* and are available exclusively on nightly because their behavior is still under active design and soundness evaluation. Although nightly enables powerful experimentation, it provides no stability guarantees: features may change, be redesigned or removed without deprecation. For this reason, production-grade Rust codebases overwhelmingly target the stable channel, limiting developers from relying on certain expressive but unstable mechanisms.

3.2 Dispatch Techniques

Modern programming languages employ a variety of dispatch strategies at run-time or compile-time to determine which function or method is executed. These strategies fundamentally influence a language's expressiveness, performance guarantees and capacity for static reasoning [6, 33]. Rust integrates several dispatch paradigms to reconcile expressive abstraction with predictable performance, adhering to the principle of *zero-cost abstraction* [42, 44]. Most decisions are resolved statically, enabling the compiler to generate optimized machine code without runtime overhead [2, 14]. At the same time, Rust offers controlled forms of dynamic dispatch where late binding is essential, such as trait objects [31]. Beyond these core mechanisms, techniques like inlining heuristics [8], function cloning [15] and trait-based overloading [45] enable the compiler to further optimize code based on contextual information [10, 49]. Together, Rust's dispatch mechanisms provide a flexible yet robust model that preserves safety and performance.

Monomorphization Monomorphization [21] is the process by which the compiler generates concrete, type-specific versions of generic functions, methods and data structures. When a generic function is instantiated with a particular type parameter, the compiler produces a specialized version of the function in the final binary. This approach mirrors C++ template instantiation [46], enabling aggressive optimizations such as inlining, constant propagation and dead-code elimination [2, 49, 14].

```cpp
template <typename T>
T add(T a, T b) {
    return a + b;
}
int main() {
    // 1. Instantiation with int
    int x = add<int>(5, 3);
    // 2. Instantiation with double
    double y = add<double>(5.5, 3.2);
    return 0;
}
```

```cpp
int add_int(int a, int b) {
    return a + b;
}
double add_double(double a, double b) {
    return a + b;
}
int main() {
    int x = add_int(5, 3);
    double y = add_double(5.5, 3.2);
    return 0;
}
```

Listing 9. *Monomorphization process in C++: on the left, a generic function `add<T>` is defined and instantiated with two different types (`int` and `double`); on the right, the compiler generates two specialized versions of the function (`add_int` and `add_double`) and uses them accordingly.*

Because all type information is known at compile time, monomorphized functions avoid virtual dispatch overhead and allow the compiler to reason precisely about memory layout [29]. The primary trade-off is *code bloat*: large applications with many generic instantiations may experience increased binary size. Nevertheless, Rust's monomorphization strategy is central to its performance model, making generic abstractions effectively as fast as handwritten, type-specific code.

Overloading Overloading allows multiple functions or methods to share the same name but differ in parameter types or counts. Rust does not support traditional function overloading by parameter types as seen in languages like C++ or Java. Instead, it employs a unified mechanism based on *trait resolution* and *associated functions* [26]. Overloading emerges indirectly: different types may provide different implementations of the same trait methods and users may introduce trait bounds to guide method selection. This form of ad hoc polymorphism resembles *type classes* in Haskell [48, 22], where method resolution is determined by the combination of the type and the trait. Rust enforces coherence to guarantee that the compiler can unambiguously select an implementation at compile time [11, 18], preventing conflicts caused by independently defined overlapping instances. In addition, traits can define associated functions that behave similarly to static methods, allowing for API designs that mimic overloading while remaining type-safe and deterministic. This approach ensures predictable dispatch behavior, maintains clarity in method resolution and avoids the complexities of multi-parameter or ambiguous overload sets common in other languages [23].

Specialization Specialization is an advanced dispatch mechanism that allows more specific trait implementations to refine or override default behavior. In Rust, specialization aims to resolve long-standing expressiveness gaps in the trait system by enabling prioritized selection among overlapping implementations. This is particularly useful when a general-purpose implementation exists (often expressed as a *default* method in a trait) but certain types require optimized or semantically distinct behavior [30]. The feature builds on Rust's *coherence* and *orphan rules*[39], ensuring that overlapping implementations maintain type-safety and global consistency [11]. Although as of today specialization remains unstable and experimental (through the `#![feature(specialization)]` or `#![feature(min_specialization)]` flags [37, 38]), it provides a foundation for advanced library design patterns, such as optimized container types, numerical traits and fallback-based APIs [30]. The mechanism is conceptually related to multi-methods and ad hoc polymorphism in other languages [9, 7], but Rust restricts its form to preserve determinism and avoid ambiguity in resolution that have yet to be fully addressed.

```cpp
template <typename T>
void myMethod(T value) {
    cout << "Generic implementation" << endl;
}

template <>
void myMethod<int>(int value) {
    cout << "Specialized for int" << endl;
}

int main() {
    myMethod(10);       // prints "Specialized for int"
    myMethod(10.5);     // prints "Generic implementation"
    return 0;
}
```

Listing 10. *Specialization in C++. The code is the equivalent of that in Listing 2.*

```rust
#![feature(specialization)]
trait MyTrait {
    fn my_method(&self);
}
impl<T> MyTrait for T {
    default fn my_method(&self) {
        println!("Generic implementation");
    }
}
impl MyTrait for i32 {
    fn my_method(&self) {
        println!("Specialized for i32");
    }
}
fn main() {
    10.my_method();     // prints "Specialized for i32"
    10.5.my_method();   // prints "Generic implementation"
}
```

Listing 11. *Specialization in Rust (unstable). The code is the equivalent of that in Listing 10.*

3.3 Metaprogramming

Metaprogramming refers to the practice of writing code that generates or manipulates other code, enabling abstraction, automation and domain-specific optimizations. Historically rooted in Lisp's symbolic macro systems [32] and later expanded through compile-time techniques such as C++ templates [46], metaprogramming has evolved into a fundamental paradigm in modern language design. Its goals are multifaceted: reducing boilerplate, enforcing structural invariants, encoding repetitive patterns and enabling compile-time specialization or reasoning that would otherwise be impossible or inefficient at run-time. Across programming ecosystems, metaprogramming serves both practical and expressive purposes, ranging from code generation and DSL construction to advanced type-level computation. Rust approaches metaprogramming through a powerful and expressive macro system designed to maintain the language's guarantees of safety, determinism and predictable compilation. Rust distinguishes between two primary categories: *declarative macros* and *procedural macros*.

Declarative Macros Declarative macros (defined using `macro_rules!`) operate using pattern matching over token streams, allowing developers to define custom syntactic constructs that expand into valid Rust code. While reminiscent of Lisp macros in spirit, Rust's declarative macros enforce *hygiene* [27, 28, 13]: macro-generated identifiers cannot inadvertently capture variables or introduce unintended bindings. This prevents entire classes of errors common in textual or non-hygienic macro systems.

Procedural Macros Procedural macros provide direct access to the compiler's abstract syntax tree (AST [2, 4]) through the `proc_macro` API. These macros behave as compiler

plugins that transform annotated code items at compile time. Three major forms exist: *function-like procedural macros*, *attribute macros* and *derive macros*. The latter are widely used to automatically implement traits such as `Clone`, `Debug` or `Serialize`, significantly reducing repetitive boilerplate and enabling library authors to design ergonomic declarative APIs. Attribute macros allow developers to attach custom semantics or transformations to items, facilitating the creation of embedded domain-specific languages (DSLs) and compile-time validation tools; the `#[when(...)]` attribute macro introduced in this thesis is an example of such usage. Function-like procedural macros (e.g. the `spec!` macro) enable arbitrary code generation based on input token streams, allowing for complex compile-time computations and code synthesis. Because procedural macros operate on structured syntax trees instead of raw text, they can enforce well-formedness and preserve strong static guarantees during expansion.

Integration with the Compilation Pipeline Rust's macro system integrates deeply with the compiler's type checking and optimization phases. Unlike languages where macros expand at a separate preprocessing stage (e.g. the C preprocessor [17]), Rust expands macros as part of the front-end compilation pipeline, ensuring that hygiene, scoping and type resolution remain consistent. This design enables robust static analyses to occur alongside macro expansion and avoids undesired ambiguities or semantic inconsistencies. The combination of hygienic rewriting, AST-level transformations and compile-time type awareness makes Rust's metaprogramming capabilities both expressive and safe, empowering developers to construct efficient abstractions without compromising reliability.

4
A deep dive into MyTool

4.1 Introduction

In Chapter 2, we presented a practical example showcasing the usage of MyTool. This chapter dives deeper into its design principles and implementation details, elucidating the underlying mechanisms that enable stable specialization through metaprogramming. The actual implementation is in Rust, but as stated earlier, the concepts can be generalized to other programming languages with similar capabilities. First, we begin by outlining the core design goals that guided the development of the tool.

Design Goals The primary design goals for MyTool were centered around achieving effective specialization in stable code while adhering to the principles of safety, usability and performance. These goals shaped the tool's architecture, macro design and code generation techniques. The key objectives included:

- **Stable Compatibility**: Ensure that the tool operates entirely within the confines of stable Rust, avoiding any reliance on nightly features or unstable APIs.
- **Type Safety**: Maintain Rust's strong type safety guarantees, ensuring that specialized implementations are correctly typed and validated at compile time.
- **Ease of Use**: Provide a straightforward and intuitive interface for developers to define specialized implementations without excessive boilerplate or complexity.
- **Performance Efficiency**: Generate specialized code that is optimized for performance, minimizing overhead and ensuring that the benefits of specialization are realized in practice.

Overview As already seen, MyTool leverages the macro system to enable specialization. The three primary components needed by the approach are:

- **Conditions Definition**: A macro that annotates specialized implementations, specifying the conditions under which they should be used and generates the necessary boilerplate.
- **Method Wrapping**: A second macro that wraps method calls, directing them to the appropriate specialized implementation.
- **Build Script**: A way to analyze the code at compile time and collect the necessary information to facilitate specialization.

4.2 Conditions Definition

The first step to achieve specialization through metaprogramming is to define various specialized implementations that should be applied when certain conditions are met. In MyTool, this is accomplished using the `#[when(predicate)]` attribute macro, which takes as argument the predicate, which is the condition that must be satisfied for the specialized implementation to be selected. Please note that if we wanted a simpler drop-in replacement for the unstable specialization feature, we could edit the tool to avoid passing the predicate altogether and just use normal rust syntax for implementations, but we still would need to mark the implementation with some kind of macro to generate the necessary boilerplate.

Predicates In our implementation, the macro supports these predicates:

- **Type Equality**: Check if a generic type parameter is of a specific type, e.g. `T = i32`.
- **Trait Implementation**: Check if a generic type parameter implements a specific trait or multiple traits, e.g. `T: Display` or `T: Debug + Clone`.
- **Logical And**: Check if all the inner predicates are satisfied, e.g. `all(predicate1, predicate2, ...)`.
- **Logical Or**: Check if at least one of the provided predicates is satisfied, e.g. `any(predicate1, predicate2, ...)`.
- **Logical Not**: Check if the provided predicate is not satisfied, e.g. `not(predicate)`.

These predicates can be combined to form complex conditions, allowing for flexible and powerful specialization rules.

Wildcards In addition to the specific predicates, MyTool supports a wildcard condition that matches any type. For instance, the condition `T = Vec<_>` matches any vector type, regardless of its element type. Another useful wildcard condition is `T = &_`, which matches any reference type.

Default Implementation In addition to specialized implementations, MyTool supports a default implementation that is used when no other conditions are met. These default implementations are picked up by default by the Build Script and don't need to be explicitly marked with any macro. Optionally, if we wanted to limit the implementations available for specialization, we could use another macro to explicitly mark the default implementations.

Normalization To ensure that the conditions are consistent and can be easily compared, MyTool normalizes the conditions. This normalization process involves transforming the conditions into a canonical form, which allows for easier comparison and resolution of overlapping conditions. The normal form chosen is a disjunctive normal form (DNF) [36], where each condition is represented as a conjunction of predicates. This means that each condition is a logical OR of one or more predicatesand each

predicate is a logical AND of one or more atomic conditions. The normalization process also involves applying De Morgan's laws [16] to push negations down to the atomic conditions, ensuring that the conditions are expressed in a consistent manner. This representation allows for efficient checking of whether a condition is satisfied, because we can easily check each disjunctive clause separately. For example, the condition `not(all(T: Display, any(T = i32, T = f64)))` is normalized to `any(not(T: Display), all(not(T = i32), not(T = f64)))`, so we know that that specific specialized implementation should be applied either if `T` does not implement `Display` or if `T` is neither `i32` nor `f64`.

Macro expansion Whenever possible, the conditions are evaluated at compile time and the generic parameters are replaced or constrained accordingly in all occurrences. For example: if we have a type equality condition, the generic parameter is replaced with the specific type; if we have a trait implementation condition, the generic parameter is constrained to implement the specified trait(s). Given that all the conditions are normalized, the macro expansion can be greatly optimized, allowing for efficient resolution of the conditions at compile time. Each disjunct condition is expanded into its own `trait` and `impl` couple, where the new trait's name is originated by appending the conditions hash to the original trait name. This is the reason why we previously defined this approach *monomorphic specialization*. For ease of read, in this thesis we will give each specialized trait a name that represents the underlying conditions, instead of the actual names the tool would output.

Nested Conditions The macro also supports nested conditions, allowing for more complex specialization rules. For example, we can have a condition that checks if a type is a vector of types that implement a specific trait, e.g. `all(T = Vec<U>, U: Display)`. Some particular cases could also be completely flattened during normalization, e.g. `all(T = Vec<U>, any(U = i32, U = f64))` would be flattened to `any(T = Vec<i32>, T = Vec<f64>)` and hence an implementation would be generated for each of the two cases.

Examples Listings 13 and 14 show an example of how to define specialized implementations using the `#[when(predicate)]` macro and the corresponding expansion generated by the macro. The examples generate specializations for a struct `MyType` and a trait `Foo`, which takes a generic type parameter `T` and has only one method `foo(&self, x: T)`, as seen in Listing 12.

```
1 struct MyType;
3 trait Foo<T> {
4     fn foo(&self, x: T);
5 }
```

Listing 12. *Definition of the simple struct and trait used in the following listings.*

```rust
// Default implementation
impl<T> Foo<T> for MyType {
    fn foo(&self, x: T) {
        println!("Default Foo for MyType");
    }
}

// Specialized implementation for Display
#[when(T: Display)]
impl<T> Foo<T> for MyType {
    fn foo(&self, x: T) {
        println!("Foo for MyType where T implements Display");
    }
}

// Specialized implementation for &[_]
#[when(T = &[_])]
impl<T> Foo<T> for MyType {
    fn foo(&self, x: T) {
        println!("Foo for MyType where T is &[_]");
    }
}

// Specialized implementation for not Vec<_>
#[when(not(T = Vec<_>))]
impl<T> Foo<T> for MyType {
    fn foo(&self, x: T) {
        println!("Foo for MyType where T is not Vec<_>");
    }
}

// Specialized implementation for MyI8 or &u8
type MyI8 = i8;

#[when(any(T = MyI8, T = &u8))]
impl<T> Foo<T> for MyType {
    fn foo(&self, x: T) {
        println!("Foo for MyType where T is MyI8 or &u8");
    }
}
```

Listing 13. *Example of the conditions definition using the tool.*

```rust
// Default implementation
impl<T> Foo<T> for MyType {
    fn foo(&self, x: T) {
        println!("Default Foo for MyType");
    }
}

// Specialized implementation for Display
trait FooDisplay {
    fn foo(&self, x: String);
}

impl<T: Display> FooDisplay<T> for MyType {
    fn foo(&self, x: T) {
        println!("Foo for MyType where T implements Display");
    }
}

// Specialized implementation for &[_]
trait FooSlice<T> {
    fn foo(&self, x: &[T]);
}

impl<T> FooSlice<T> for MyType {
    fn foo(&self, x: &[T]) {
        println!("Foo for MyType where T is &[_]");
    }
}

// Specialized implementation for not Vec<_>
trait FooNotVec<T> {
    fn foo(&self, x: T);
}

impl<T> FooNotVec<T> for MyType {
    fn foo(&self, x: T) {
        println!("Foo for MyType where T is not Vec<_>");
    }
}

// Specialized implementation for MyI8 or &u8
type MyI8 = i8;

trait FooMyI8 {
    fn foo(&self, x: MyI8);
}

impl FooMyI8 for MyType {
    fn foo(&self, x: MyI8) {
        println!("Foo for MyType where T is MyI8 or &u8");
    }
}

trait FooRefU8 {
    fn foo(&self, x: &u8);
}

impl FooRefU8 for MyType {
    fn foo(&self, x: &u8) {
        println!("Foo for MyType where T is MyI8 or &u8");
    }
}
```

Listing 14. *Example of the expansion of the conditions definition defined in Listing 13.*

4.3 Method Wrapping

To enable the selection of the appropriate specialized implementation, we need a second macro that wraps method calls. In MyTool, this is accomplished using the `spec!` macro, which takes as first arguments the method call itself.

Type Annotations If the language supported reflection, if we had some way to inspect the type of the arguments at compile time or if this was implemented directly in the compiler, the method call alone would have been enough. That not being the case for Rust, we need to provide additional information to the macro. This additional pieces of information include the types of object on which the method is called and the types of the arguments passed to the method, if any.

- **Type of the Object**: The type of the object on which the method is called, e.g. if we have `x.fn(arg1, arg2)`, we need to provide the type of `x`, e.g. `MyType`.
- **Types of the Arguments**: A list of the types of the arguments passed to the method, if the method takes no arguments, it can be empty or omitted, e.g. if we have `x.fn(arg1, arg2)`, we need to provide the types of `arg1` and `arg2`, e.g. `[Arg1Type, Arg2Type]`.

With this information, the tool can solve almost all of the specialization at compile time, generating a direct call to the appropriate specialized implementation.

Annotations The *type annotations* defined above are the first two of what we call *annotations*, which are additional pieces of information provided to the macro to help it resolve the specialization. In Rust, just like we can't inspect the type of a variable at compile time, there is also no way to inspect the traits implemented by a type or to know type aliases. To overcome these limitations, MyTool allows the user to provide additional annotations to the method wrapping macro (once again notice that if the language supported reflection or similar features this would not be necessary, even more so if specialization was implemented directly in the compiler). These additional annotations include:

- **Implemented Traits**: A list of traits implemented by each relevant type, e.g. `i32: Display` or `i32: Debug + Clone`.
- **Type Aliases**: A list of type aliases for each relevant type, e.g. `MyInt = i32`.

These annotations can be provided as additional arguments to the `spec!` macro, allowing the tool to accurately resolve the specialization even in the presence of type aliases or trait bounds in the conditions. Considering all of this, the full syntax for the `spec!` macro is the following: `spec!{ method_call; var_type; args_types; annotations... }`, where `method_call` is the method call, `var_type` is the type of the object on which the method is called, `args_types` is a list of the types of the arguments passed to the method (if any) and `annotations` are any additional annotations provided by the user. Additionally, for languages that support class inheritance, other annotations could be added to reflect hierarchy, e.g. `T1 <: T2`.

Overlap An important aspect to consider when implementing specialization is the possibility of overlapping conditions. When multiple specialized implementations could apply to a given method call, the tool needs a way to resolve the overlap and select the most appropriate implementation. One way to handle this is to define a priority system based on the specificity of the conditions. For example, a condition that checks for type equality (e.g. `T = i32`) could be considered more specific than a condition that checks for trait implementation (e.g. `T: Display`). In MyTool, this is effectively how overlaps are resolved, just like in Rust's unstable specialization feature. When multiple conditions of the same priority level overlap, the tool raises a compile-time error, prompting the user to resolve the ambiguity by refining the conditions. A more general and broader mechanism is the *lattice* rule defined in Rust original specialization RFC [37], which instead of requiring a strict priority system, allows partial ordering given that there is also an additional implementation that covers precisely the area of overlap. Though this approach is more flexible, it is also more complex to implement and since this is not the main priority of this thesis, we opted for the simpler priority system. The chain of priority used in MyTool is as follows:

1. Type Equality
2. Trait Implementation
3. Not Type Equality
4. Not Trait Implementation

$$\text{T = \&i32} > \text{T = \&U} \geq \text{T = \&_} > \text{T: Display + Debug} > \text{T: Display} > \text{not(T = \&i32)} > \text{not(T = \&U)} \geq \text{not(T = \&_)} > \text{not(T: Display + Debug)} > \text{not(T: Display)}$$

Listing 15. *Example of priority ordering of conditions, highest to lowest. U is a generic type parameter and hence overlaps with the wildcard unless it is constrained in some other conditions.*

Examples Listings 16 and 17 show an example of how to wrap method calls using the `spec!` macro and the corresponding expansion generated by the macro. The examples use the specialized implementations defined in Listings 13 and 14.

```
fn main() {
    let x = MyType;
    spec! { x.foo(vec![1, 2, 3]); MyType; [Vec<i32>] };
    spec! { x.foo("hello".to_string()); MyType; [String]; String: Display };
    spec! { x.foo(&[1, 2, 3]); MyType; [&[i32]] };
    spec! { x.foo("hello"); MyType; [&str] };
    spec! { x.foo(1i8); MyType; [i8]; MyI8 = i8 };
    spec! { x.foo(&1u8); MyType; [&u8] };
}
```

Listing 16. *Example of method wrapping, using implementations defined in Listing 13.*

```rust
fn main() {
    let x = MyType;
    <MyType as Foo<Vec<i32>>>::foo(&x, vec![1, 2, 3]);
    <MyType as FooDisplay>::foo(&x, "hello".to_string());
    <MyType as FooSlice<i32>>::foo(&x, &[1, 2, 3]);
    <MyType as FooNotVec<&str>>::foo(&x, "hello");
    <MyType as FooMyI8>::foo(&x, 1i8);
    <MyType as FooRefU8>::foo(&x, &1u8);
}
```

Listing 17. *Example of the expansion of method wrapping from Listing 16.*

```
Default Foo for MyType
Foo for MyType where T implements Display
Foo for MyType where T is &[_]
Foo for MyType where T is not Vec<_>
Foo for MyType where T is MyI8 or &u8
Foo for MyType where T is MyI8 or &u8
```

Listing 18. *Example of the output printed by the program from Listing 16.*

4.4 Build Script

The final step is to correctly handle specialization no matter the order in which the various components are defined. Without this step, if a specialized implementation is defined after a method call site that should use it, the macro would not be able to resolve the specialization correctly. As seen in Listing 4, to solve this issue with our implementation of MyTool we simply need to add one line to the `build.rs` file. With that, the build script analyzes the code at compile time and collects all the necessary information to handle specialization correctly. This build script scans the code for all the `#[when(predicate)]` attributes and collects the specialized implementations, as well as the default implementations and the traits implemented by each relevant type. We've also implemented a mechanism to find the *conditions definition* macros even if they are imported with a different name, by analyzing the macro definitions in the code, e.g. `use mytool::when as my_when;`. With this information, the build script generates a metadata file that is later used by the macros to resolve the method calls correctly.

Alternatives If the language didn't support build scripts or similar mechanisms, an alternative approach could be to use some additional macros to mark the default implementations and the traits explicitly. The first is required to identify the default implementations, while the second is needed because the trait's body is used to generate the monomorphized versions of the specialized implementations. To handle the order of definitions without a build script or similar mechanism, the easiest way would be to have a two-pass compilation process, where the first pass collects all the necessary information and the second pass generates the specialized code. However, this approach would be less efficient and more complex to implement, as it would require significant changes to the compilation process. Another way would be to wrap everything in a macro that collects all the necessary information, but this would be cumbersome and

would not scale well for larger codebases.

4.5 Lifetimes

One of the main issues the Rust team faced when stabilizing specialization was handling lifetimes correctly. Lifetimes are a core feature of Rust's ownership system and ensuring that specialized implementations respect lifetimes is crucial for maintaining safety guarantees and soundness of the type system. It was indeed due to these challenges that specialization got stalled for a long time before being eventually marked as unstable. When designing MyTool, we had to carefully consider how to handle the matter.

Soundness Concerns The main soundness concern with lifetimes and specialization arises when a specialized implementation requires, implicitly or explicitly, different lifetime constraints than those present in the default implementation. For example, if a specialized implementation returns a reference with a shorter lifetime than the default implementation, it could lead to dangling references or other unsafe behavior. The core problem is that at code generation time (the phase where MIR is lowered to LLVM IR and then to machine code) the compiler doesn't know about lifetimes (since it has already erased all lifetime information) and so it can't use them to choose which implementation should apply. This code (as shown in Listings 19 and 20) would compile with `#![feature(specialization)]` but lead to unsound behavior.

```
1  trait MyTrait {
2      fn my_method(&self);
3  }
4  impl<T> MyTrait for T {
5      default fn my_method(&self) {
6          println!("Generic implementation");
7      }
8  }
9  impl<T: 'static> MyTrait for T {
10     fn my_method(&self) {
11         println!("Specialized for 'static");
12     }
13 }
```

Listing 19. *Example of a soundness issue with explicit lifetime constraint.*

Workarounds This soundness issue was reported shortly after the specialization proposal [1] and stalled its progress for a long time. To mitigate the issue, the Rust team proposed a workaround that allows sound specialization for a subset of use cases [38]. This solution, called `min_specialization`, is unfortunately a breaking change from current Rust code and that's one of the reasons why it didn't get stabilized. For example, the code in Listings 19 and 20 would not compile with `#![feature(min_specialization)]`.

```rust
trait MyOtherTrait {}
impl<T: 'static> MyOtherTrait for T {}

trait MyTrait {
    fn my_method(&self);
}
impl<T> MyTrait for T {
    default fn my_method(&self) {
        println!("Generic implementation");
    }
}
impl<T: MyOtherTrait> MyTrait for T {
    fn my_method(&self) {
        println!("Specialized for 'static");
    }
}
```

Listing 20. *Example of a soundness issue with implicit lifetime constraint.*

Our Approach As stated earlier, the fundamental problem is that the compiler doesn't know about lifetimes at code generation time even though they can affect which implementation should be selected. The most sensible solution to this would be in our opinion to make the compiler "smarter" by simply retaining lifetime information until code generation time, even though this would require significant changes to the compiler. This ides has already been brought up in the past but it was discarded as it was considered to have a "high cost" [1]. The only other accusation against it is that it would be harder to reason about lifetimes if they could influence code selection, but we believe that this is not a strong argument against it, given that many modern programming languages have already accustomed us to using generic programming features without significant issues. Hence, in MyTool we decided to simulate the information that the "smarter" compiler would have about lifetimes by allowing specialization based on lifetimes in *conditions definition* and providing lifetime annotations in *method wrapping*.

Conditions Definition To allow the user to define specialized implementations based on lifetimes, we extended the `#[when(predicate)]` macro to support lifetime predicates. These predicates include:

- **Type Equality**: When defining a type equality with a reference type, we can also specify lifetime constraints, e.g. `T = &'a i32`.
- **Lifetime Bound**: Check if a generic type parameter has a specific lifetime, e.g. `T: 'b`.

Other predicates could be combined with these to form more complex conditions:

- **Lifetime Equality**: Check if a lifetime parameter is equal to a specific lifetime, e.g. `'a = 'static`.
- **Lifetime Outlives**: Check if a lifetime parameter outlives another lifetime, e.g. `'a: 'b`.

Just like for the other conditions defined earlier, also these new conditions generate explicit constraints in the expanded implementations.

Method Wrapping To provide the necessary lifetime information to the method wrapping macro, we extended the `spec!` macro to accept lifetime annotations. First of all, in *type annotations* reference types can now include lifetime information, e.g. `&'a i32`. Additionally, we added a new type of annotation to specify the lifetimes of each relevant type:

- **Lifetime Bound**: The lifetimes of each relevant type, e.g. `T: 'a`.

Furthermore, we could add a new type of annotation to specify lifetime relationships:

- **Lifetime Relationships**: Specify lifetimes equality or outlives relationships, e.g. `'a: 'b` or `'a = 'static`.

```
trait MyTrait {
    fn my_method(&self);
}
impl<T> MyTrait for T {
    fn my_method(&self) {
        println!("Generic implementation");
    }
}
#[when(T: 'static)] // or #[when(T = &'static _)]
impl<T> MyTrait for T {
    fn my_method(&self) {
        println!("Specialized for 'static");
    }
}
fn call_method(x: &'static i32, y: &i32) {
    spec! { x.my_method(); &'static i32 } // prints "Specialized for 'static"
    spec! { y.my_method(); &i32 }         // prints "Generic implementation"
}
```

Listing 21. *Example of specialization using lifetimes.*

4.6 Limitations

While MyTool provides a powerful mechanism for stable specialization, it does have some limitations that should be considered:

- **Boilerplate**: The need for additional annotations in method wrapping can introduce some boilerplate, especially in complex scenarios with many types and traits involved.
- **Compile-Time Overhead**: The use of macros and build scripts can introduce some compile-time overhead, especially in large codebases with many specialized implementations.

- **Lifetimes Complexity**: While the tool provides mechanisms to handle lifetimes, the added complexity may lead to confusion or misuse if not properly understood by the user.
- **Trait Bounds**: Since the tool generates a monomorphic version of each trait involved, the ability to involve those traits in trait bounds elsewhere in the code is lost.
- **Limited Predicate Support**: The current set of supported predicates may not cover all language feature (e.g. `dyn Trait`), potentially limiting the tool's applicability in certain cases.

Despite these limitations, MyTool represents a significant step forward in enabling stable specialization through metaprogramming, providing developers with a powerful tool to optimize their code while adhering to Rust's safety guarantees. More generally, the concepts presented in this chapter can be adapted to other programming languages with similar capabilities, broadening the applicability of the approach beyond Rust.

5 Experimentation

5.1 Introduction

To validate the utility of stable specialization, we conducted a series of experiments focusing on ecosystem analysis. These experiments aimed to identify real-world scenarios where stable specialization could provide significant benefits, particularly in terms of performance and code maintainability. The experiments were designed to analyze a diverse set of public Rust codebases, examining their use of traits and generic programming patterns. By applying our specialization technique to these codebases, we sought to demonstrate its practical applicability and effectiveness in real-world scenarios.

Approach To conduct the experiments, we developed a static analysis tool that scans the selected Rust codebases for instances of the identified patterns, which we'll show later. More specifically, we instructed the compiler to build a custom AST starting from the HIR for each function (including trait and impls fn items) in the codebase.

After building the AST, using an heuristic we grouped together functions that could be specializable between one another. To do so, we used a naming-based heuristic combined with a same-number-of-parameters check. For example, if we had a function named `process_int` taking an `i32` parameter and another function named `process_float` taking an `f32` parameter, we would group them together as they likely represent specialized versions of a more generic function. We also split the functions based on their origin (bare function, trait items, impl items) and if they were associated with a trait, we used another similar heuristic for the two traits.

Once we had these groups, we calculated the tree edit distance of each function in the group to one another. This was done using the ZSS algorithm [50], which computes the minimum number of edit operations required to transform one tree into another, allowing us to quantify the structural similarity between functions' ASTs. By analyzing these distances, we could identify groups of functions that were structurally similar enough to be considered for specialization using our stable specialization technique. After calculating the edit distance, we got the similarity by using this formula

$$1 - \frac{\text{dist}}{\text{max_size}}$$

where `dist` is the tree edit distance between the two functions and `max_size` is the size of the largest AST between the two.

Finally, we filtered out groups that had an average similarity lower than a certain threshold to focus on the most promising candidates for specialization. We also made sure to add a further optimization to avoid analyzing trees that had way too different sizes, as they would mathematically never reach a good enough similarity score, using the following formula

$$\frac{\text{min_size}}{\text{max_size}} < \text{threshold}$$

where `min_size` is the size of the smallest AST between the two and `threshold` is the similarity threshold set for the analysis.

This approach allowed us to systematically identify potential specialization opportunities across a wide range of Rust codebases, providing a solid foundation for evaluating the effectiveness of stable specialization in real-world scenarios. It's important to note that our analysis was based on heuristics to identify the relevant patterns and the specialization opportunities within the selected codebases. As such, there is a possibility that some patterns were overlooked or misclassified, which could affect the comprehensiveness of our findings. It's also important to note that not all instances where specialization is possible it should be applied, e.g. if the specialized versions do not provide significant performance improvements or if they complicate the code unnecessarily, like adding more boilerplate than they would reduce.

Project Selection An important step to conduct this research was to select a set of representative Rust projects from the open-source ecosystem. For this purpose, we selected some of the most downloaded crates from `crates.io` as well as some popular projects and other less known ones, ranging from code editors to broadly used libraries. The selected projects (over 50 in total) cover a wide range of domains and use cases, ensuring that our analysis captures a diverse set of programming patterns and practices.

Pattern Identification To keep the analysis focused and feasible and to ensure that we could draw meaningful conclusions from the results, we concentrated on identifying specific patterns that are likely to benefit from stable specialization. These patterns include:

- **Manual Dispatch**: In the trait we have a version of the function for each type we want to support and in the implementation we manually dispatch to the correct version based on the type of the input.
- **Monomorphized Traits**: A trait is defined for each type we want to support and each implementation provides a monomorphized version of the function.
- **Standalone Functions**: A standalone function is defined for each type we want to support and the caller is responsible for calling the correct version based on the type of the input.
- **Inherent Impls**: A struct has inherent implementations for each type we want to support, providing specialized versions of the function.

Other than these, we've also pointed out some additional patterns that we did not include in the analysis because they were less common or harder to analyze in a systematic way, such as:

- **Type Level Dispatch**: A type-level mechanism is used to select the appropriate implementation at compile time, this does not support default implementations.
- **Macros**: Macros are used to generate specialized implementations based on the input type, this reduces boilerplate but does not support default implementations.
- **Pattern matching blocks**: Pattern matching is used to select the appropriate implementation based on the type of the input.
- **If blocks**: Conditional statements are used to select the appropriate implementation based on the type of the input.

Many of these patterns can be seen in the code examples in Listings 24 to 28 and the baseline implementations using unstable specialization and MyTool specialization can be seen in Listings 22 and 23 respectively. Each pattern represents a different approach to achieving specialization in Rust and our analysis aimed to determine how stable specialization could enhance each of these approaches. It's important to note that even though each of these patterns could be implemented using stable specialization, some patterns are more likely to benefit from it than others. Anyways, we believe that stable specialization could provide significant benefits across all these patterns by reducing boilerplate code, improving performance and enhancing code readability and maintainability. Furthermore, many of these patterns do not support certain specialization features, like specialization based purely on trait bounds or default implementations. By leveraging stable specialization, developers can achieve more efficient and maintainable code across a variety of programming patterns.

```rust
#![feature(specialization)]
struct MyType;
trait Foo<T> {
    fn foo(&self, v: T);
}
impl<T> Foo<T> for MyType {
    default fn foo(&self, v: T) {
        println!("default");
    }
}
impl Foo<i32> for MyType {
    fn foo(&self, v: i32) {
        println!("v is i32");
    }
}
```

Listing 22. *Unstable Specialization.*

```rust
struct MyType;
trait Foo<T> {
    fn foo(&self, v: T);
}
impl<T> Foo<T> for MyType {
    fn foo(&self, v: T) {
        println!("default");
    }
}
#[when(T = i32)]
impl<T> Foo<T> for MyType {
    fn foo(&self, v: T) {
        println!("v is i32");
    }
}
```

Listing 23. MyTool *Specialization.*

```rust
struct MyType;
fn foo_default<T>(x: &MyType, v: T) {
    println!("default");
}
fn foo_i32(x: &MyType, v: i32) {
    println!("v is i32");
}
```

Listing 24. *Standalone Functions.*

```rust
struct MyType;
impl MyType {
    fn foo_default<T>(&self, v: T) {
        println!("default");
    }
    fn foo_i32(&self, v: i32) {
        println!("v is i32");
    }
}
```

Listing 25. *Inherent Impls.*

```rust
struct MyType;
trait Foo<T> {
    fn foo_default(&self, v: T);
    fn foo_i32(&self, v: i32);
}
impl<T> Foo<T> for MyType {
    fn foo_default(&self, v: T) {
        println!("default");
    }
    fn foo_i32(&self, v: i32) {
        println!("v is i32");
    }
}
```

Listing 26. *Manual Dispatch.*

```rust
struct MyType;
trait FooDefault<T> {
    fn foo(&self, v: T);
}
trait FooI32 {
    fn foo(&self, v: i32);
}
impl FooDefault for MyType {
    fn foo(&self, v: T) {
        println!("default");
    }
}
impl FooI32 for MyType {
    fn foo(&self, v: i32) {
        println!("v is i32");
    }
}
```

Listing 27. *Monomorphized Traits.*

```rust
struct MyType {}
struct MyTypeWithV<T> {
    my_type: MyType,
    v: T,
}
trait Foo {
    fn foo(&self);
}
impl Foo for MyTypeWithV<i32> {
    fn foo(&self) {
        println!("v is i32");
    }
}
impl Foo for MyTypeWithV<u32> {
    fn foo(&self) {
        println!("v is u32");
    }
}
```

Listing 28. *Type Level Dispatch.*

5.2 Results

The results of our experimentation revealed several interesting insights into the prevalence and effectiveness of stable specialization in real-world Rust codebases. Across the analyzed projects, we identified numerous instances where stable specialization could be applied to reduce boilerplate code and improve performance. In particular, we found that the **TODO** and **TODO** patterns were the most common, indicating that many developers resort to these approaches to achieve specialization in their code. By applying stable specialization to these patterns, we observed significant reductions in code complexity and improved maintainability. Furthermore, our analysis showed that stable specialization could effectively handle default implementations and specialization based purely on trait bounds, which are not supported by some of the existing patterns. This capability allows developers to write more generic and reusable code without sacrificing performance. As expected the major results are in projects that heavily utilize traits and generics, as these are the areas where stable specialization is expected to have the most impact. Overall, our experimentation demonstrated that stable specialization is a valuable tool for Rust developers, offering a practical solution to common challenges associated with generic programming and code specialization. The analysis was run twice: once with a similarity threshold of 0.9 and once with a similarity threshold of 0.99. The results of these analyses are summarized in Table **??** and Table **??** respectively.

Header 1	Header 2	Header 3	Header 4	Header 5	Header 6	Header 7
Data A1	Data B1	Data C1	Data D1	Data E1	Data F1	Data G1
Data A2	Data B2	Data C2	Data D2	Data E2	Data F2	Data G2

Table 5.1. *Results with similarity threshold of 0.9.*

Header 1	Header 2	Header 3	Header 4	Header 5	Header 6	Header 7
Data A1	Data B1	Data C1	Data D1	Data E1	Data F1	Data G1
Data A2	Data B2	Data C2	Data D2	Data E2	Data F2	Data G2

Table 5.2. *Results with similarity threshold of 0.99.*

6
Related Work

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

7
Future Work

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

8 Conclusions

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Bibliography

[1] Aaron Turon. Shipping specialization: a story of soundness. `https://aturon.github.io/blog/2017/07/08/lifetime-dispatch/`, 2017.

[2] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Pearson/Addison-Wesley, 2nd edition, 2006.

[3] Roland Backhouse, Patrik Jansson, Johan Jeuring, and Lambert Meertens. Generic programming. In S. Doaitse Swierstra, José N. Oliveira, and Pedro R. Henriques, editors, *Advanced Functional Programming*, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.

[4] I.D. Baxter, A. Yahin, L. Moura, M. Sant'Anna, and L. Bier. Clone detection using abstract syntax trees. In *Proceedings. International Conference on Software Maintenance (Cat. No. 98CB36272)*, pages 368–377, 1998.

[5] Chandrasekhar Boyapati, Robert Lee, and Martin Rinard. Ownership Types for Safe Programming: Preventing Data Races And Deadlocks. In Satoshi Matsuoka, editor, *Proceedings of the 17th Annual ACM Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA'02)*, pages 211–230, Seattle, WA, USA, November 2002. ACM Press.

[6] Luca Cardelli and Peter Wegner. On understanding types, data abstraction, and polymorphism. In *ACM Computing Surveys*, volume 17, 1985.

[7] Giuseppe Castagna. Covariance and contravariance: Conflict without a cause. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 17(4), 1995.

[8] J. Cavazos and M.F.P. O'Boyle. Automatic tuning of inlining heuristics. In *SC '05: Proceedings of the 2005 ACM/IEEE Conference on Supercomputing*, pages 14–14, 2005.

[9] Craig Chambers, David Ungar, and Elgin Lee. An efficient implementation of self, a dynamically-typed object-oriented language based on prototypes. In *OOPSLA '92*, 1992.

[10] B. F. Chang and L. K. John. Exploring the limits of inlining. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 1997.

[11] Adam Chlipala. Certified programming with dependent types. In *Lecture Notes*, 2011. MIT Course Notes, early chapters discuss type class coherence and overlapping instance safety.

[12] David G Clarke, John M Potter, and James Noble. Ownership types for flexible alias protection. In Craig Chambers, editor, *Proceedings of 13th International Conference*

on Object-Oriented Programming Systems, Languages and Applications (OOPSLA'98), pages 48–64, Vancouver, BC, CAnada, October 1998. ACM.

[13] William D. Clinger and Mitchell Wand. Hygienic macro technology. *Proceedings of the ACM on Programming Languages*, 4, 2020.

[14] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 13(4), 1991.

[15] Dibyendu Das. Function inlining versus function cloning. *SIGPLAN Not.*, 38(6), June 2003.

[16] A. De Morgan. *Formal Logic: The Calculus of Inference, Necessary and Probable*. University Press of the Pacific, 2003.

[17] Christian J. Dietrich. Cppsig: Extracting type information for c-preprocessor macro expansions. *Proceedings of the 11th Workshop on Programming Languages and Operating Systems*, 2021.

[18] Jacques Garrigue. Programming with polymorphic variants. In *Proceedings of the International Conference on Functional Programming (ICFP)*, 1998.

[19] Jean-Yves Girard. Linear Logic. *Theoretical Computer Science*, 50(1):1–101, 1987.

[20] Jean-Yves Girard, Yves Lafont, and Laurent Regnier. *Advances in Linear Logic*. Cambridge University Press, July 1995.

[21] Robert Griesemer, Raymond Hu, Wen Kokke, Julien Lange, Ian Lance Taylor, Bernardo Toninho, Philip Wadler, and Nobuko Yoshida. Featherweight go. *Proceedings of the ACM on Programming Languages*, 4:1 – 29, 2020.

[22] Cordelia Hall, Kevin Hammond, Simon L. Peyton Jones, and Philip Wadler. Type classes in haskell. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 18(2), 1996.

[23] Simon Peyton Jones, Mark Jones, and Erik Meijer. Type classes: An exploration of the design space. In *Proceedings of the ACM SIGPLAN Haskell Workshop*, 2000.

[24] Kyo C. Kang, Sholom G. Cohen, James A. Hess, William E. Novak, and A. Spencer Peterson. Feature-Oriented Domain Analysis (FODA) Feasibility Study. Technical Report CMU/SEI-90-TR-21, Carnegie Mellon University, Pittsburgh, Pennsylvania, USA, November 1990.

[25] ahmet serkan karataş, halit oğuztüzün, and ali doğru. global constraints on feature models. In david cohen, editor, *proceedings of the 16th international conference on principles and practice of constraint programming (cp'10)*, lncs 6308, pages 537–551, st. andrews, scotland, September 2010. springer.

[26] Steve Klabnik and Carol Nichols. The rust programming language. `https://doc.rust-lang.org/book/`, 2019.

[27] Eugene Kohlbecker, Daniel P. Friedman, Matthias Felleisen, and Bruce Duba. Hygienic macro expansion. In *Proceedings of the ACM SIGPLAN Symposium on Interpreters and Interpretive Techniques*. ACM, 1992.

[28] Eugene E. Kohlbecker, Daniel P. Friedman, Matthias Felleisen, and Bruce F. Duba. Hygienic macro expansion. In *LFP '86*, 1986.

[29] Chris Lattner and Vikram Adve. Llvm: A compilation framework for lifelong program analysis and transformation. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2004.

[30] Nicholas D. Matsakis. Rfc 1210: Specialization. `https://github.com/rust-lang/rfcs/pull/1210`, 2015.

[31] Nicholas D. Matsakis and Felix S. Klock II. The rust language. In *Proceedings of the ACM SIGAda Annual Conference on High Integrity Language Technology (HILT)*, 2014.

[32] John McCarthy. Recursive functions of symbolic expressions and their computation by machine, part i. *Communications of the ACM*, 3(4), 1960.

[33] Robin Milner. A theory of type polymorphism in programming. *Journal of Computer and System Sciences*, 17(3), 1978.

[34] David R. Musser and Alexander A. Stepanov. Generic programming. In P. Gianni, editor, *Symbolic and Algebraic Computation*, Berlin, Heidelberg, 1989. Springer Berlin Heidelberg.

[35] Martin Odersky. Observers for Linear Types. In Bernd Krieg-Brückner, editor, *Proceedings of the 4th European Symposium on Programming (ESOP'92)*, LNCS 582, pages 390–407, Rennes, France, February 1992. Springer.

[36] Emil L. Post. Introduction to a general theory of elementary propositions. *American Journal of Mathematics*, 43(3):163–185, 1921.

[37] Rust Project Developers. Specialization rfc. `https://rust-lang.github.io/rfcs/1210-impl-specialization.html`, 2015.

[38] Rust Project Developers. Maximally minimal specialization: always applicable impls. `https://smallcultfollowing.com/babysteps/blog/2018/02/09/maximally-minimal-specialization-always-applicable-impls/`, 2018.

[39] Rust Project Developers. The orphan rules. `https://doc.rust-lang.org/reference/items/implementations.html#orphan-rules`, 2024. Accessed: 2025-01-01.

[40] Rust Project Developers. Rust release channels. `https://doc.rust-lang.org/book/appendix-07-nightly-rust.html`, 2024. Accessed: 2025-01-01.

[41] Christoph Seidl, Tim Winkelmann, and Ina Schaefer. A Software Product Line of Feature Modeling Notations and Cross-Tree Constraint Languages. In Andreas Oberweis and Ralf H. Reussner, editors, *Proceedings of the 13th Edition of Modellierung (Modellierung'16)*, LNI P-254, pages 157–172, Karlsruhe, Germany, March 2016. Springer.

[42] Guy L. Steele. Making asynchronous parallelism safe for the world. In *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '90, New York, NY, USA, 1989. Association for Computing Machinery.

[43] Bjarne Stroustrup. *The Design and Evolution of C++*. Addison-Wesley, first edition, March 1994.

[44] Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley, 3rd edition edition, 2005.

[45] Peter J. Stuckey and Martin Sulzmann. A theory of overloading. *ACM Trans. Program. Lang. Syst.*, 27(6), November 2005.

[46] David Vandevoorde and Nicolai M. Josuttis. *C++ Templates: The Complete Guide*. Addison-Wesley, 2002.

[47] Philip Wadler. Linear Types Can Change the World! In Manfred Broy and Cliff B. Jones, editors, *Proceedings of the 2nd Working Conference on Programming Concepts and Methods (IFIP'90)*, pages 561–582, Sea of Galilee, Israel, April 1990. North-Holland.

[48] Philip Wadler and Stephen Blott. How to make ad-hoc polymorphism less ad-hoc. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 1989.

[49] Mark N. Wegman and F. Kenneth Zadeck. Constant propagation with conditional branches. In *POPL '85*. ACM, 1985.

[50] Kaizhong Zhang and Dennis Shasha. Simple Fast Algorithms for the Editing Distance Between Trees and Related Problems. *SIAM Journal on Computing*, 1989.